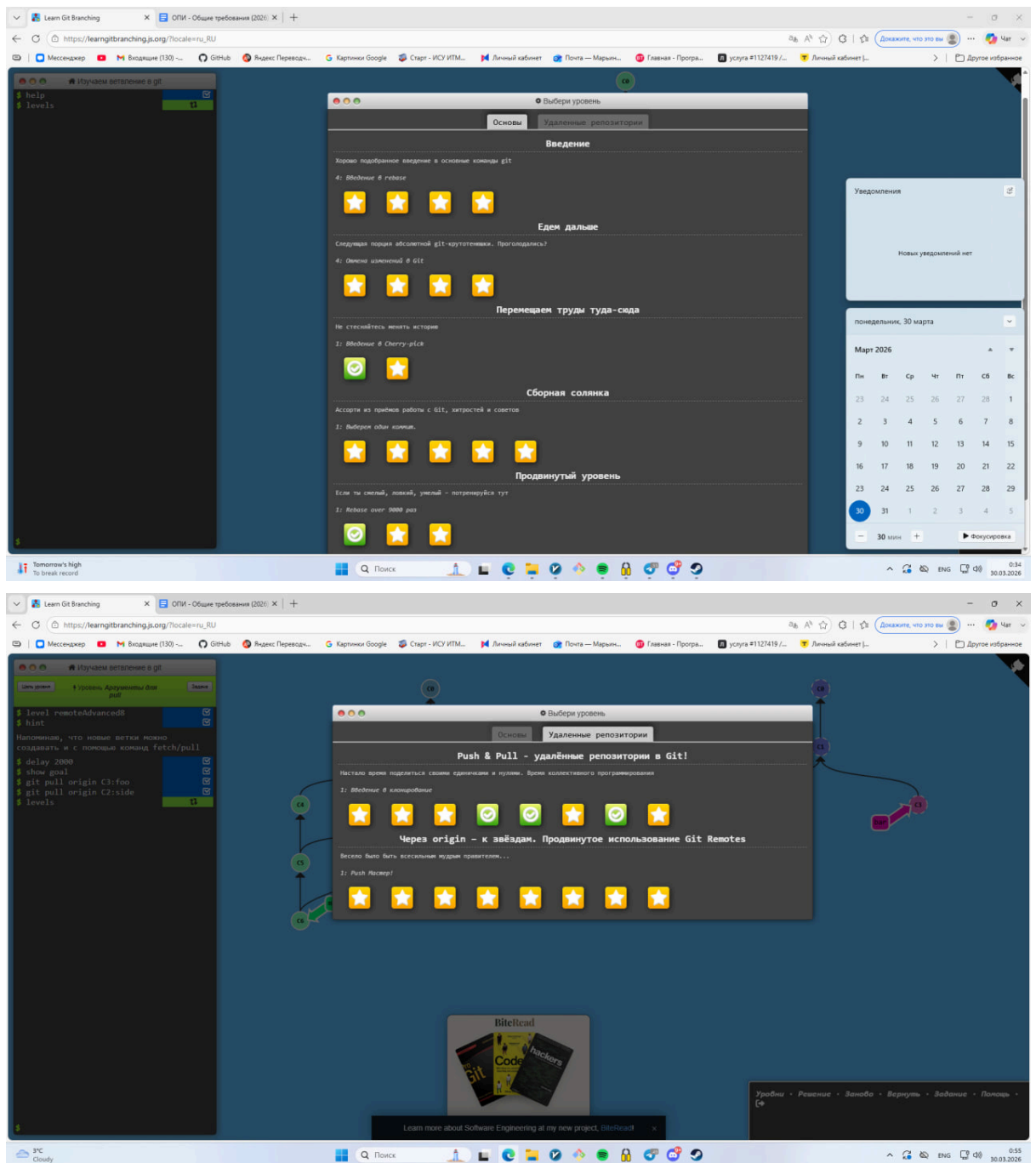
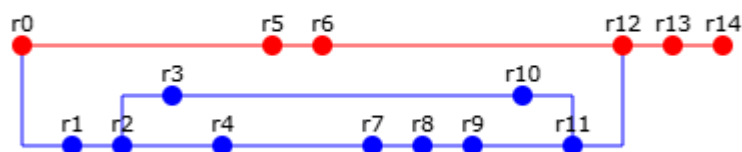




1. Задание se.ifmo:

Вариант 68351



Сконфигурировать в своём домашнем каталоге репозитории git и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий git, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

```
git init
git config user.name "red"
git config user.email "red@red.com"

cd ..
unzip -o src/commit0.zip -d OpiLab2
cd OpiLab2/

git checkout -b red
git add .
git commit -m "r0"

cd ..
unzip -o src/commit1.zip -d OpiLab2
cd OpiLab2/

git checkout -b blue
git add .
git commit --author="blue <blue@blue.com>" -m "r1"

cd ..
unzip -o src/commit2.zip -d OpiLab2
cd OpiLab2/

git checkout -b blue
git add .
git commit --author="blue <blue@blue.com>" -m "r2"

git checkout -b blue2
cd ..
unzip -o src/commit3.zip -d OpiLab2
cd OpiLab2/
```

```
git add .
git commit --author="blue <blue@blue.com>" -m "r3"

git checkout blue
cd ..
unzip -o src/commit4.zip -d OpiLab2
cd OpiLab2/
git add .
git commit --author="blue <blue@blue.com>" -m "r4"

git checkout red

cd ..
unzip -o src/commit5.zip -d OpiLab2
cd OpiLab2/
git add .
git commit -m "r5"

cd ..
unzip -o src/commit6.zip -d OpiLab2
cd OpiLab2/

git add .
git commit -m "r6"

git checkout blue

cd ..
unzip -o src/commit7.zip -d OpiLab2
cd OpiLab2/

git add .
git commit --author="blue <blue@blue.com>" -m "r7"

cd ..
unzip -o src/commit8.zip -d OpiLab2
cd OpiLab2/

git add .
git commit --author="blue <blue@blue.com>" -m "r8"

cd ..
unzip -o src/commit9.zip -d OpiLab2
cd OpiLab2/
```

```
git add .
git commit --author="blue <blue@blue.com>" -m "r9"

git checkout blue2

cd ..
unzip -o src/commit10.zip -d OpiLab2
cd OpiLab2/

git add .
git commit --author="blue <blue@blue.com>" -m "r10"

git checkout blue
git merge --no-commit blue2

git add .

cd ..
unzip -o src/commit11.zip -d OpiLab2
cd OpiLab2/
git add .
git commit --author="blue <blue@blue.com>" -m "r11"

git checkout red
git merge --no-commit blue

git add .

cd ..
unzip -o src/commit12.zip -d OpiLab2
cd OpiLab2/
git add .
git commit -m "r12"

cd ..
unzip -o src/commit13.zip -d OpiLab2
cd OpiLab2/
git add .
git commit -m "r13"

cd ..
unzip -o src/commit14.zip -d OpiLab2
cd OpiLab2/
git add .

git commit -m "r14"
```

2. Работа с удаленным репозиторием.

2.1 Опубликовал работу на удаленный репозиторий (github)

```
git remote add origin https://github.com/wwwyssa/Opilab2.git
```

2.3

Разница между командами.

Рассмотрим команду **git fetch**.

```
git log --oneline --graph --all
* 7ff81a9 (HEAD -> main, origin/main) r14
* e55ad20 r13
* b861a7f r12
|
| * c22db01 (origin/blue, blue) r11
| |
| | * 108099c (origin/blue2, blue2) r10
| | * e6ca58d r3
| | * c2e8c80 r9
| | * 760b3da r8
| | * 2add958 r7
| | * b003880 r4
| |
| |
| * 83d468b r2
| * 2ea44d3 r1
* | 9881fc3 r6
* | 6b4dc23 r5
|/
* 06c39e0 r0
```

Вот структура до команды fetch.

Затем я добавил комментарий в файл A.java на другом устройстве и запустил на удаленный репозиторий. Затем использовал git fetch origin

```
$ git log --oneline --graph --all
* 9894fb7 (origin/main) Добавил с компа коммент //add comment in A.java в A.java
* 7ff81a9 (HEAD -> main) r14
* e55ad20 r13
* b861a7f r12
|
| * c22db01 (origin/blue, blue) r11
| |
| | * 108099c (origin/blue2, blue2) r10
| | * e6ca58d r3
| | * c2e8c80 r9
| | * 760b3da r8
| | * 2add958 r7
| | * b003880 r4
| |
| |
| * 83d468b r2
| * 2ea44d3 r1
* | 9881fc3 r6
* | 6b4dc23 r5
|/
* 06c39e0 r0
```

Видно, что ветка origin/main находится впереди локальной ветки main, которая указывает на старый коммит. Чтобы применить прилетевшие изменения к соевой ветке я использую git merge origin/main

```
$ git merge origin/main
Updating 7ff81a9..9894fb7
Fast-forward
 A.java | 1 +
 1 file changed, 1 insertion(+)

grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git log --oneline --graph --all
* 9894fb7 (HEAD -> main, origin/main) Добавил с компа коммент //add comment in A.java в A.java
* 7ff81a9 r14
* e55ad20 r13
* b861a7f r12
\
* c22db01 (origin/blue, blue) r11
| \
| * 108099c (origin/blue2, blue2) r10
| * e6ca58d r3
* | c2e8c80 r9
* | 760b3da r8
* | 2add958 r7
* | b003880 r4
|/
* 83d468b r2
* 2ea44d3 r1
* | 9881fc3 r6
* | 6b4dc23 r5
|/
* 06c39e0 r0
```

Рассмотрим команду **git pull**

Еще раз изменю файл, но в этот раз D.java и запущу изменения на удаленный репозиторий, но в этот раз применяем команду git pull

```
grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 425 bytes | 106.00 KiB/s, done.
From https://github.com/wwwyssa/OPI-lab-2
 * branch          main          -> FETCH_HEAD
 * cee32a1..5f89042 main        -> origin/main
Updating cee32a1..5f89042
Fast-forward
 D.java | 1 +
 1 file changed, 1 insertion(+)

grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git log --oneline --graph --all
* 5f89042 (HEAD -> main, origin/main) Добавил с компа коммент //add comment in D.java в D.java еще раз
* cee32a1 Добавил с компа коммент //add comment in D.java в D.java
* 9894fb7 Добавил с компа коммент //add comment in A.java в A.java
* 7ff81a9 r14
* e55ad20 r13
* b861a7f r12
\
* c22db01 (origin/blue, blue) r11
| \
| * 108099c (origin/blue2, blue2) r10
| * e6ca58d r3
* | c2e8c80 r9
* | 760b3da r8
* | 2add958 r7
* | b003880 r4
|/
* 83d468b r2
* 2ea44d3 r1
* | 9881fc3 r6
* | 6b4dc23 r5
|/
* 06c39e0 r0
```

Видно, что git выполнил сразу две команды git fetch + git merge.

Получается, что `git fetch` загружает изменения из удаленного репозитория, но не объединяет их с локальными. Рабочая директория остается нетронутой. В тоже время `git pull` меняет файлы в рабочей директории.

git push

`git push` - команда, отправляющая локальные коммиты на удаленный сервер.

- `git push <server_name> <branch>`
- `-u` - флаг, связывающий локальную ветку с веткой на `origin`.
- `-f` - перезаписывает ветку на сервере, на ту что локально
- `--all` - пушит все ветки

git stash, git switch

```
grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ echo "123" >> A.java

grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   A.java

no changes added to commit (use "git add" and/or "git commit -a")

grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git stash
warning: in the working copy of 'A.java', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on main: 5f89042 Добавил с компа коммент //add comment in D.java в D.java еще раз

grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git stash list
stash@{0}: WIP on main: 5f89042 Добавил с компа коммент //add comment in D.java в D.java еще раз
```

`git stash` - команда, которая позволяет на время “архивировать” изменения, чтобы их можно было применить позже.

- `git stash pop` - позволяет применить ранее отложенные изменения и удалить их из стека
- `git stash apply` - позволяет применить ранее отложенные изменения не удаляя их из стека
- `git stash -u` - позволяет отложить неотслеживаемые файлы
- `git stash -a` - позволяет отложить все файлы, в тч из `.gitignore`

git switch

`git switch` - команда для переключения между ветками. Ее отличие от `git checkout` в том, что она не такая многофункциональная. Если `git checkout` мог работать с файлами, например `git checkout -- t.txt` (откат файла к состоянию из последнего коммита), с конкретными коммитами, то `git switch` работает исключительно с ветками.

- `git switch <ветка>` - переместить на ветку
- `git switch -` - переместиться на ветку, на которой были до последнего перехода.
- `git switch --detach <hash>` - переключение на коммит, но ветка не создается. В этом режиме можно как бы просматривать код, но если сделать коммиты, то они не будут принадлежать никакой ветке

git submodule

git submodule - это ссылка на конкретный коммит в другом репозитории

Создадим еще один репозиторий. Я назвал его Submodule-for-OPI-lab-2. В нем просто какой-то один файл Module.java. Затем я добавил его в свой репозиторий, как сабмодуль.

- git submodule add <url> <path> - добавление сабмодуля. Создает .gitmodules, скачивает содержимое в указанную папку.
- git submodule update - переход сабмодуля к последнему коммиту
- --remote - обращение к удаленному серверу
- --merge - автоматический мержд при подкачке.

\$ git submodule add https://github.com/wwwyssa/Submodule-for-OPI-lab-2.git

```
grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitmodules
    new file:   Submodule-for-OPI-lab-2
```

Видно, что появились новые файлы. В .gitmodules указаны внешние репозитории, подключенные к данному.

```
grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ cat .gitmodules
[submodule "Submodule-for-OPI-lab-2"]
  path = Submodule-for-OPI-lab-2
  url = https://github.com/wwwyssa/Submodule-for-OPI-lab-2.git
```

Я добавил коммит в Submodule-for-OPI-lab-2. Значит, что сейчас репозиторий ссылается не на последнюю версию сабмодуля.

```
grisa@LAPTOP-UVES55EQ MINGW64 /e/OpiLab2/OpiLab2 (main)
$ git submodule update --remote --merge
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 271 bytes | 54.00 KiB/s, done.
From https://github.com/wwwyssa/Submodule-for-OPI-lab-2
  4a3cdbb..07b6302  master    -> origin/master
Updating 4a3cdbb..07b6302
Fast-forward
  Module.java | 1 +
  1 file changed, 1 insertion(+)
Submodule path 'Submodule-for-OPI-lab-2': merged in '07b630267311c36adb26ec5d6a74c509b90ec499'
```

Обновили сабмодуль.


```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Submodule-for-OPI-lab-2 (new commits)

no changes added to commit (use "git add" and/or "git commit -a")
```

Теперь мы синхронизировали сабмодуль с основным проектом.

Смоделируем конфликт, связанный с различными версиями подмодуля.

В сабмодуле сделал вторую ветку branch1. В удаленном репозитории была ссылка на ветку main, в локальном на branch1 и когда я попробовал сделать git pull, то произошел конфликт

```
$ git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 2 (delta 1), reused 2 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 231 bytes | 19.00 KiB/s, done.
From https://github.com/wwwyssa/OPI-lab-2
   83dc7da..e3ecbd0  main       -> origin/main
Failed to merge submodule Submodule-for-OPI-lab-2 (commits don't follow merge-base)
CONFLICT (submodule): Merge conflict in Submodule-for-OPI-lab-2
Recursive merging with submodules currently only supports trivial cases.
Please manually handle the merging of each conflicted submodule.
This can be accomplished with the following steps:
- go to submodule (Submodule-for-OPI-lab-2), and either merge commit 07b6302
  or update to an existing commit which has merged those changes
- come back to superproject and run:

    git add Submodule-for-OPI-lab-2

  to record the above merge or update
- resolve any other conflicts in the superproject
- commit the resulting index in the superproject
Automatic merge failed; fix conflicts and then commit the result.
```

Решил конфликт тем, что локально тоже переключился на ветку master

git hooks

Это встроенный механизм, который позволяет механически запускать скрипты при наступлении каких-то событий.

Я написал свой хук на commit-msg. Этот хук будет запускаться после того, как напишется коммит. Он его проверит на длину, минимальная длина должна быть 10 символов, иначе коммит не создастся.

```

$ cat commit-msg
#!/bin/bash

COMMIT_MSG_FILE=$1

COMMIT_TEXT=$(cat "$COMMIT_MSG_FILE")

if [ ${#COMMIT_TEXT} -lt 10 ]; then
    echo "minimalnaya dlina soobsheniya 10 simvolov"
    exit 1
fi
echo "horoshii commit"
exit 0

```

Виды хуков:

- pre-commit - запускается первым, до того как напечатается сообщение коммита. Используется для проверки данных перед созданием коммита.
- prepare-commit-msg - запускается до вызова редактора сообщения коммита, но после создания стандартного сообщения
- commit-msg - принимает один параметр - путь до временного файла с коммитом. Если скрипт завершает работу с 0, то все хорошо. Если с 1, то коммит не будет создан
- post-commit - запускается после создания коммита.
- pre-rebase - выполняется при попытке rebase и может его отменить.
- post-checkout - запускается после git checkout
- post-merge - запускается после успешного выполнения команды merge.
- pre-push - выполняется во время команды git push. Срабатывает после проверки ссылок, но до отправки данных.
- pre-receive - запускается при старте получения данных. Он получает на stdin список отправленных изменений.
- update - похож на pre-receive, но выполняется для каждой ветки отдельно
- post-receive - выполняется после окончания процесса передачи данных.

git worktree

```

$ git worktree add ../OPI-lab-2-fix -b fix-branch
Preparing worktree (new branch 'fix-branch')
HEAD is now at 16a21f5 1

```

Создал новое worktree

```

grisa@LAPTOP-UVESS5EQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main)
$ cd ..

grisa@LAPTOP-UVESS5EQ MINGW64 /e/OpiLab2/OpiLab2-Second
$ ls
OPI-lab-2/  OPI-lab-2-fix/

```

Создал там коммит и запустил его

```

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2-fix (fix-branch)
$ echo "fix" >> A.java

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2-fix (fix-branch)
$ git add .
warning: in the working copy of 'A.java', LF will be replaced by CRLF the next time Git touches it

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2-fix (fix-branch)
$ git commit -m "fix bug"
minimalnaya dlina soobsheniya 10 simvolov

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2-fix (fix-branch)
$ git commit -m "fix bug in A.java"
horoshii commit
[fix-branch 129e13e] fix bug in A.java
1 file changed, 1 insertion(+)

```

Затем удалил worktree

```

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main)
$ git worktree remove E:/OpiLab2/OpiLab2-Second/OPI-lab-2-fix

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main)
$ cd ..

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second
$ ls
OPI-lab-2/

```

git sparse-checkout

Создал две папки

```

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main)
$ ls
A.java D.java Lab4.java Submodule-for-OPI-lab-2/ _ lab1/ lab2/

```

прописал git sparse-checkout lab1

```

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main)
$ git sparse-checkout set lab1

grisa@LAPTOP-UVES5SEQ MINGW64 /e/OpiLab2/OpiLab2-Second/OPI-lab-2 (main|SPARSE)
$ ls
A.java D.java Lab4.java Submodule-for-OPI-lab-2/ _ lab1/

```

Остались только файлы в корне, сабмодуль и lab1, lab2 пропал.

После отключения SPARSE lab2 вернулась.

b